School of Engineering · Undergraduate Research Project

# Digital Forensics Chain-of-Custody Tracker — Management Using DSA

Implementation of Linked List, Binary Search Tree, and Evidence Tree structures for forensic chain-of-custody verification

**PRESENTED BY**

Joe Jelson J · E25B000152

Ayush Rai · E25B000195

**UNDER**

**School of Engineering**

**Ajeenkya DY Patil University**

Project Domain · Cybersecurity / Digital Forensics · DSA

# Digital Forensics

## Chain-of-Custody Tracker

DSA Project using Linked List + Binary Search Tree + Evidence Tree

### Linked List

Chronological, append-only chain of custody

### BST

Fast hash-based evidence lookup

### Tree + Recursion

Nested files, folders and archives

# 1. Problem Statement

- Digital investigations produce many evidence items such as disk images, logs, emails and documents.
- Investigators need to preserve the order in which evidence was collected and transferred.
- Evidence should be searchable by its digital fingerprint (hash).
- Nested folders and archives are hierarchical and need recursive traversal.
- This project demonstrates practical applications of Data Structures and Algorithms.

# 2. Project Objectives

### Maintain custody

Store evidence events in chronological order.

### Verify integrity

Compare stored and current hashes.

### Search efficiently

Use BST operations for hash lookup.

### Explore hierarchy

Represent folders and archives as a tree.

### Demonstrate DSA

Apply pointers, recursion, searching and traversal.

### Usable interface

Provide a simple menu-driven workflow.

# 3. System Architecture

### Evidence Input

File name, type, hash, timestamp, investigator

### Linked List

Chronological chain-of-custody records

### BST

Hash → Evidence ID lookup

### Evidence Tree

Folder/archive hierarchy

### Verification

Stored hash vs current hash

# 4. Linked List — Chain of Custody

```
struct Evidence {
    int id;
    string name;
    string type;
    string hash;
    string timestamp;
    string investigator;
    Evidence* next;
};
```

## Why Linked List?

Each new custody event can be appended without shifting existing records.

## Core Operations

append • display • search by ID

## Complexity

Append with tail pointer: O(1). Search by ID: O(n).

# 5. Binary Search Tree — Hash Lookup

```
struct BSTNode {
    string hash;
    int evidenceID;
    BSTNode* left;
    BSTNode* right;
};
```

## Insert

Place each hash according to its ordering.

## Search

Compare target hash and move left or right.

## Complexity

Average balanced BST: O(log n); worst case: O(n).

# 6. Evidence Tree + Recursion

```
void displayTree(Node* root) {
    if (root == nullptr) return;

    cout << root->name << endl;

    for (Node* child : root->children)
        displayTree(child);
}
```

## Example

Laptop → Downloads → Suspicious.zip → files

## Traversal

Recursion visits every nested child.

## Complexity

Tree traversal: O(n), where n is number of nodes.

# 7. Hash-Based Integrity Verification

- Generate a cryptographic fingerprint for each evidence file, preferably SHA-256.
- Store the original hash with the custody record.
- On verification, calculate the current hash again.
- If the values match → integrity check passes.
- If they differ → report a possible modification or corruption.

```
Stored SHA-256  ==  Current SHA-256

       YES → INTEGRITY VERIFIED
       NO  → POSSIBLE TAMPERING
```

# 8. Program Workflow

1 Add evidence

2 Create custody node

3 Store hash in BST

4 Attach to evidence tree

5 Search / verify

6 Display report

# 9. Menu-Driven Interface

```
========================================
 DIGITAL FORENSICS CHAIN-OF-CUSTODY
========================================
1. Add Evidence
2. Display Chain of Custody
3. Search Evidence by Hash
4. Display Evidence Tree
5. Verify Integrity
6. BST Traversal
7. Exit

Enter choice:
```

# 10. Sample Output

```
CHAIN OF CUSTODY

[101] laptop_image.dd
Type: Disk Image
Hash: 8A91BC72...
Time: 09:42 AM

        ↓

[102] browser_history.log
Type: Log
Hash: 72BC91AA...
Time: 09:48 AM

SEARCH HASH: 8A91BC72...
✓ Evidence Found
Integrity: VERIFIED
```

# 11. Time & Space Complexity

| Operation | Typical / Average | Worst / Extra |
|---|---|---|
| Linked List append | O(1) with tail | O(n) search |
| Linked List search | O(n) | O(1) extra |
| BST search | O(log n) avg. | O(n) worst |
| BST insertion | O(log n) avg. | O(n) worst |
| Evidence tree traversal | O(n) | O(h) stack |
| Hash calculation | O(file size) | Implementation dependent |

# 12. Advantages & Limitations

## Advantages

- Multiple DSA concepts in one application.
- Clear chronological custody trail.
- Fast average hash lookup with BST.
- Recursive handling of nested evidence.
- Easy to extend into GUI/database.

## Limitations

- Normal BST can become unbalanced.
- Academic demo is not certified forensic software.
- Real systems need secure storage and access control.
- Hash matching detects changes but does not identify the person who made them.

# 13. Future Enhancements

- Replace BST with AVL Tree or Red-Black Tree for guaranteed O(log n) search.
- Add a graphical user interface for investigation workflows.
- Store records in a secure database with role-based access control.
- Export forensic reports to PDF/CSV.
- Support SHA-256 and SHA-512 verification.
- Add digital signatures and immutable audit logging.
- Add unit tests and larger simulated evidence datasets.

# 14. Conclusion

- The project models a digital forensic chain of custody using fundamental DSA techniques.
- Linked List preserves the chronological evidence trail.
- BST enables efficient hash-based searching.
- Evidence Tree and recursion model nested folders and archives.
- Hash comparison provides a practical integrity-verification mechanism.
- Together, these structures demonstrate DSA applied to cybersecurity and digital forensics.

# Thank You

Questions & Discussion